Verification and Validation of Concurrent State in a Time-Bounded Auction

Md Minhajul Abedin - 202483033 | Nabil Hasan - 202488877

Project Report

ENGI 9839: Software Verification and Validation

19th July 2026

Abstract

This report covers a verification and validation (V&V) case study on AuctionEdge, a full-stack, time-bounded auction API (FastAPI, PostgreSQL, React) built around properties that are inherently concurrency-sensitive: accepting a bid, holding funds in a wallet, extending a soft-close timer, and closing an auction automatically all involve state that more than one user can touch at the same moment. Rather than relying on one testing technique, the project layers six of them — unit, property-based, contract-based, integration, concurrency, and system testing — and then checks its own thoroughness with mutation testing. Four research questions guide the work: (RQ1) does the system serialize concurrent bids correctly and resolve genuine timestamp ties; (RQ2) does the test suite actually catch boundary-comparison faults, measured by mutation score; (RQ3) does automated, generated test input turn up edge cases a hand-written boundary table would miss; and (RQ4) how does application-level pessimistic locking compare to database-level `serializable` isolation under identical concurrent load. The resulting suite comprises 188 automated tests, 99% branch coverage on the system's core logic, and a 100% mutation kill rate (100/100 mutants). Testing also caught and fixed a real bug — a crash triggered by `NaN`/`Infinity` input — found specifically by HTTP-boundary fuzzing, and a separate round of exploratory testing against the live frontend turned up two further, purely UI-side defects that none of the backend-focused tiers could have seen.

1 Introduction

Verifying a concurrent system is difficult for a simple reason: the properties that matter most — no lost updates, no double-spending, no race condition at a deadline — are exactly the ones a single test run cannot observe. An ordinary unit test can show that a function returns the right value for a given input. It cannot show what happens when two callers hit that function at the same instant against the same shared, persistent state. This project was built to work on that gap directly: rather than choosing one verification technique and applying it well, it layers six distinct testing techniques against a single system and treats what each one finds as its own, separately measurable result.

The subject of this case study, AuctionEdge, is a from-scratch implementation of a time-bounded auction platform where the current price is visible but Buy-It-Now and reserve thresholds are not revealed until they matter. Its core use cases — placing a bid, retracting one within a fixed window, an automatic system-triggered close, and a wallet subsystem that must never let a user's committed holds exceed their balance — are all standard instances of the concurrent-state problem that shows up in real e-commerce, ticketing, and financial systems. The premise behind this project is that these use cases break exactly where naive testing is weakest: the moment two bids land within milliseconds of each other, or a user tries to commit funds to two different auctions at once.

The rest of this report is organized as follows. Section 2 covers the literature behind the techniques used. Section 3 describes the system under test. Section 4 lays out the methodology, including the four research questions that structure the investigation. Section 5 describes how each testing tier was implemented. Section 6 presents results, organized by research question. Section 7 discusses what was actually found, including four genuine defects discovered during testing. Section 8 covers limitations. Section 9 concludes.

The complete implementation — source code, 188 automated tests, configuration, and a step-by-step run guide — is included with this submission, alongside raw output captured

from an actual execution of the suite (Section 6 and the accompanying figures).

2 Literature Review

Mutation testing. DeMillo, Lipton, and Sayward [1] first proposed deliberately injecting small faults into a program and checking whether a test suite notices them, as a way to judge test-data quality rather than program correctness itself. Jia and Harman's survey [2] traces three decades of work since, including the specific technique this project leans on — comparison-operator mutation (e.g. `>` $\leftrightarrow$ `>=`). That mutation matters because it produces exactly the kind of boundary fault that statement or branch coverage alone will not catch: a mutant that flips a comparison operator frequently still satisfies the same coverage numbers as the original code. RQ2 in this project is a direct, applied version of the adequacy question these works raise in the abstract.

Property-based and automated test generation. The original plan for this project called for symbolic execution. In practice, property-based testing via Hypothesis was used instead — the practical descendant of Claessen and Hughes's QuickCheck [3]. QuickCheck's central idea is that a developer states an invariant once, and a generator searches a large, automatically-shrunk input space for a counterexample. That fits a dynamically-typed, I/O-adjacent Python codebase better than symbolic execution in the tradition of King [4], which needs either static, side-effect-free code or a heavier constraint-solving setup than a typical web API's business-logic layer calls for. MacIver et al.'s tool paper [5] documents the library actually used for RQ3.

Concurrency control and isolation levels. RQ4 compares application-level pessimistic locking (`SELECT ... FOR UPDATE`) against database-level `SERIALIZABLE` isolation, grounded in the transaction-processing literature. Gray and Reuter [6] give the standard treatment of locking-based concurrency control and deadlock avoidance, which directly motivated this project's fixed lock-ordering rule for transactions touching more than one wallet.

Berenson et al.'s critique of ANSI SQL isolation levels [7] explains why `SERIALIZABLE` produces abort-and-retry behaviour where a locking scheme instead blocks and proceeds — a difference this project's RQ4 experiment reproduces directly rather than merely citing.

Exploratory and taxonomy-driven testing. The exploratory "tours" run against this project follow Whittaker's tour-based taxonomy of exploratory testing [8], which treats exploratory testing as a structured, mission-driven activity rather than unguided poking around. The more general testing-technique background — equivalence partitioning, boundary value analysis, and decision-table testing — follows standard treatments in Myers et al. [9] and Ammann and Offutt [10], the latter of which also supplies the coverage-criteria vocabulary (statement, branch, and multiple-condition coverage) used throughout the white-box tier.

3 System Under Test

AuctionEdge is a FastAPI (Python) backend with a PostgreSQL persistence layer and a React frontend, implementing six core use cases: placing a bid , creating an auction listing, retracting a bid within a fixed window, viewing bid history with masked bidder identities, system-triggered auction auto-closure, and outbid/won/lost notifications. Two properties make the system a good subject for concurrency-focused V&V:

1. **The no-lost-update invariant**: when several bids arrive for the same auction in close succession, the final recorded price must equal the highest *accepted* bid, with nothing higher silently overwritten.
2. **The wallet-hold invariant**: for every user, at all times, `held_amount` $\leq$ `balance` — including under a deliberate double-spend attempt, where one user submits bids on two different auctions at the same instant with a combined value that exceeds their balance.

The codebase keeps pure business logic (`core/bidding.py`, `core/wallet.py`) separate

from database-touching orchestration (`core/wallet_db.py`, `core/auto_close.py`) and from the HTTP API layer (`api/*.py`). That split shaped the testing strategy described in Section 5: pure functions could be tested with zero I/O overhead, while the concurrency-critical paths needed real database sessions and, for one tier, real Docker containers.

4 Methodology

This project's V&V work is organized around four research questions rather than around the testing-technique list directly, on the idea that a results section built around *claims* is easier to evaluate than one built around *methods*.

- **RQ1**: Does the system correctly handle race conditions at an auction's closing moment and under simultaneous bids?
- **RQ2**: Does mutation testing on the system's boundary-comparison operators (`>` vs. `>=`) reveal any gap between test coverage and test adequacy?
- **RQ3**: Does automated, generative test-input construction surface bid and wallet edge cases beyond a hand-authored boundary-value table?
- **RQ4**: How do an application-level pessimistic locking strategy and a database-level `SERIALIZABLE` isolation strategy compare under identical concurrent contention, in terms of correctness and mechanism?

Techniques were split into tiers by cost and target, following a plan document written before implementation started (included with this submission as `AuctionEdge_Test_Plan.md`). Tier 1 techniques (white-box coverage, boundary value analysis, mutation testing, contract-based testing, property-based testing) map directly onto RQ1–RQ4. Tier 2 techniques (integration testing, equivalence partitioning, the concurrency harness) back those claims up with broader coverage. Tier 3 techniques (state-transition testing, a decision table, and exploratory testing) are deliberately light-touch, following the plan's own instruction not to over-invest in techniques whose marginal value is mostly report-traceability rather than

actually finding defects.

Table 1 summarizes the five automated testing tiers that resulted, each kept in its own directory so a tier can be run, and reasoned about, on its own.

Table 1: The five automated testing tiers compared.

Tier	Tests	Technique(s)	What it targets
`core/`	114	White-box coverage, BVA, MCC, equivalence partitioning, use-case-derived validator tests	Pure business-logic functions (`bidding.py`, `wallet.py`, `auth.py`), no I/O; the RQ2 mutation-testing oracle
`property/`	6	Property-based testing (Hypothesis)	Four core invariants across ∼100 generated examples each (RQ3)
`integration/`	59	Contract-based testing, integration testing, HTTP-boundary fuzzing	Real-DB invariants, the bid/wallet/notification chain, WebSocket broadcast, state transitions, adversarial HTTP input, and the seller-action endpoints (accept-bid, cancel, pay, relist, buy-it-now)
`concurrency/`	5	Real-thread concurrency harness, comparative analysis	Race conditions and the two-strategy locking comparison (RQ1, RQ4)
`system/`	4	System testing	The full deployed stack (Docker Compose), real HTTP, real containers
Total	**188**		184 pass unconditionally; 4 (system tier) skip automatically unless the full stack is running

Mutation testing (RQ2) is deliberately absent from Table 1: it is not a sixth pytest tier but a separate tool (`mutmut`) that re-runs the `core/` tier's own tests against 100 deliberately mutated copies of the source, making `core/` both a testing tier in its own right and the adequacy check for RQ2 (Section 5.2).

5 Implementation

5.1 White-Box Coverage, Boundary Value Analysis, and Multiple Condition Coverage

The pure-function layer (`core/bidding.py`, `core/wallet.py`) is covered by 63 unit tests, reaching 100% statement and branch coverage. Boundary values were chosen to do double duty: drive coverage now, and later serve as the mutation-testing oracle in Section 5.2. For example, the retraction window is tested at 14:59, 15:00, and 15:01 past the original bid's timestamp, and the tiered bid-increment thresholds are tested at \$49.99/\$50.00/\$50.01 and \$249.99/\$250.00/\$250.01. Where a decision contains a compound boolean condition — e.g. `auction_status != "Active" or now >= auction_end_time` — branch coverage alone only proves the *combined* condition can go both ways; it says nothing about whether `and` or `or` was used correctly. A separate multiple-condition-coverage (MCC) suite varies each sub-condition independently, so a fault like swapping `and` for `or` actually gets caught instead of hiding behind an already-green coverage report.

A further 51 tests round out the `core/` tier: 4 for `core/auth.py` (password hashing and the JWT encode/decode round trip, added specifically to close a coverage gap found partway through the project), and 47 for the Pydantic-layer validators on item categories, auction durations, and price fields (`AuctionCreate`'s starting-price, reserve-price, and buy-it-now-price rules), covered under equivalence partitioning in Section 5.7.

5.2 Mutation Testing (RQ2)

Mutation testing was run via `mutmut`, scoped to the two files containing the RQ2 boundary operators: `core/bidding.py` and `core/wallet.py`. Mutating the whole codebase would have produced an unmanageable number of mutants with no clear link back to a specific design decision; scoping it this way means every mutant maps to something concrete, like

"does a bid exactly equal to the minimum increment count as valid?" `mutmut` does not run on native Windows, so it was run under Windows Subsystem for Linux (WSL) against the same test suite. The result — 100 mutants generated (91 in `bidding.py`, 9 in `wallet.py`), all 100 killed, zero survivors — is discussed in Section 6.2.

5.3 Property-Based Testing (RQ3)

Six property-based tests, using Hypothesis, run roughly 100 automatically-generated examples each against four core functions. `validate_bid` is checked against the isolated `bid_meets_minimum` comparison across the full generated input space, not just hand-picked values. `validate_buy_it_now` is checked to never accept a price at or above the listed Buy-It-Now price. `is_retractable` is checked for monotonicity in time: once a bid stops being retractable, it must never become retractable again as time moves forward. `compute_new_end_time` is checked to never move an auction's end time earlier than it already was. Two further property tests check the wallet-hold invariant directly: for every generated (balance, held, bid amount) triple, approving a bid must never allow available balance to go negative.

5.4 Contract-Based and Integration Testing

The wallet-hold invariant and the auto-close idempotency postcondition (exactly one closure event logged per auction, even if the sweep runs twice or overlaps) were checked against a real PostgreSQL connection rather than mocked. Some of the code under test opens its own database session internally (`core/auto_close.py`), so two different test-isolation strategies were needed: tests calling functions that accept an injected session run inside a transaction that always rolls back, leaving no trace; tests calling functions that open their own session commit real rows and delete them afterward, in foreign-key-safe teardown order. Integration tests also check the full bid-placement chain (bid $\rightarrow$ wallet hold $\rightarrow$ outbid notification) and WebSocket broadcast correctness, including a check that a client watching one auction never receives another auction's events.

This tier also gained 24 tests partway through the project, covering five endpoints that had no dedicated HTTP-level test before: accepting the highest bid early, cancelling a listing, paying for a won auction, relisting an unsold item, and Buy-It-Now. These came out of a straightforward question — after adding new frontend features, did anything in the existing suite actually cover the new backend endpoints those features called? — and for these five endpoints the answer was no, so tests were added rather than left as a silent gap.

5.5 Concurrency Testing and the RQ4 Comparative Analysis

An architectural detail shaped this tier's design: `place_bid()` is declared `async def`, but every database call inside it is a plain synchronous SQLAlchemy call, and the deployment runs a single uvicorn worker with no thread-pool offload. That means concurrent HTTP requests to this endpoint get serialized by Python's own event loop before they ever reach the database — so a concurrency test that simply fires requests over HTTP, even from multiple threads, would never actually produce database-level lock contention; it would only prove that Python can queue callbacks. To get genuinely overlapping transactions, the concurrency tests instead use a `ThreadPoolExecutor` of real OS threads, each opening its own database session and calling the same internal functions `place_bid` calls, sidestepping the event-loop bottleneck entirely while still exercising the real, production locking code.

The RQ4 experiment runs an identical eight-bidder contention scenario under both the default application-level locking strategy (`SELECT ... FOR UPDATE`) and PostgreSQL's `SERIALIZABLE` isolation level, switched via a configuration flag. Results are reported in Section 6.4.

5.6 System Testing

A separate tier runs against the entire deployed system — PostgreSQL, backend, and frontend, each in its own Docker container, communicating over a real Docker network — using real HTTP requests rather than an in-process test client. This tier skips automatically when

the full stack is not running, so it never blocks the rest of the suite. Its purpose is to check behaviour that only shows up in the true deployed configuration; Section 7 discusses a genuine finding this tier alone surfaced.

5.7 Equivalence Partitioning, Decision Tables, and Exploratory Testing

Equivalence partitioning was applied to the item-category list and the auction-duration presets, both enforced at the Pydantic validation layer.

Tier 3 also includes a round of exploratory testing against the running frontend, structured as four "tours" in Whittaker's sense [8] rather than unguided clicking around: a *Standard User* tour (the full seller-to-bidder-to-payment path, watched closely on every screen rather than just checked for a 200 status), a *Saboteur* tour (double-submitting a bid, a throttled and an aborted request, refreshing mid-payment, browser back/forward after creating a listing), an *Antisocial* tour (script-injection input, an oversized photo upload, a non-image file, negative/zero/huge bid amounts, unicode and emoji text, 400% browser zoom), and an *Obsessive-Compulsive* tour (repeating the same UI action 10–45 times in a row). Each tour was driven through a real browser via Playwright automation against the actual running app. Results are reported in Section 6.6.

6 Results

6.1 Overall Suite Health

The complete automated suite comprises 188 tests across five tiers (unit, property-based, integration, concurrency, and system), all passing, at 99% branch coverage over the system's core business logic. Figure 1 shows the `core/` tier running on its own; Figure 2 shows a full run with the system tier included and coverage reported.

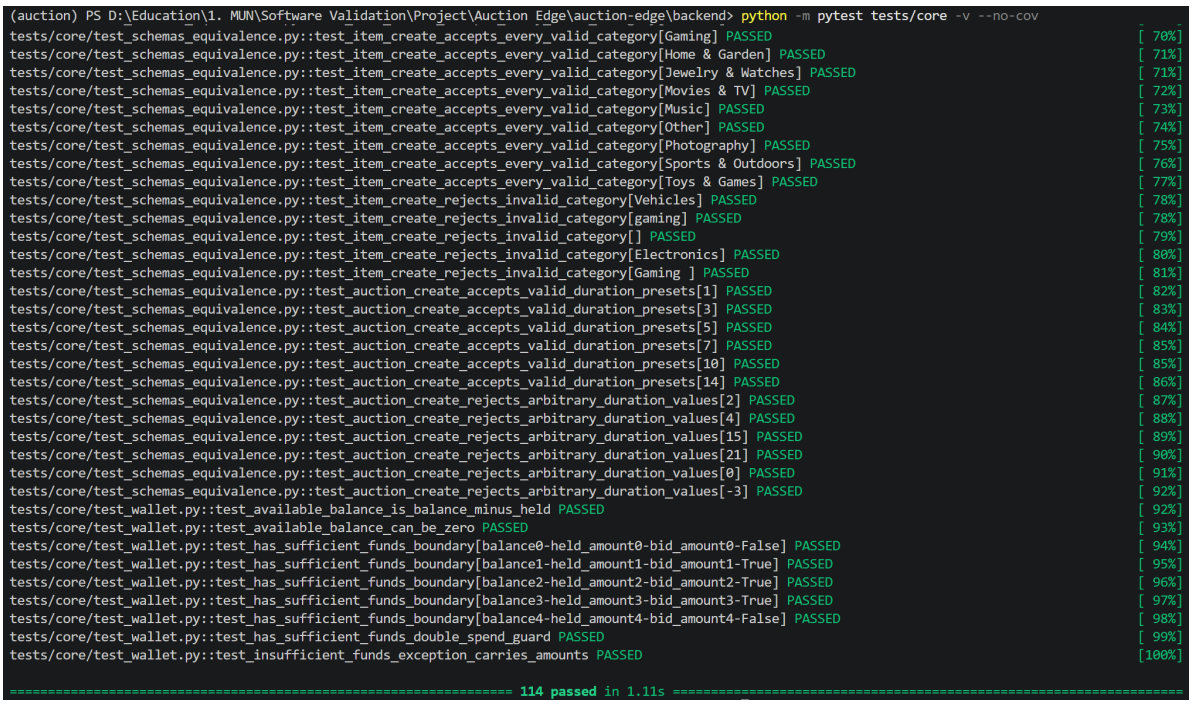

Figure 1: The `core/` tier on its own: 114 tests covering white-box, BVA, MCC, and equivalence-partitioning cases for the pure business-logic layer.

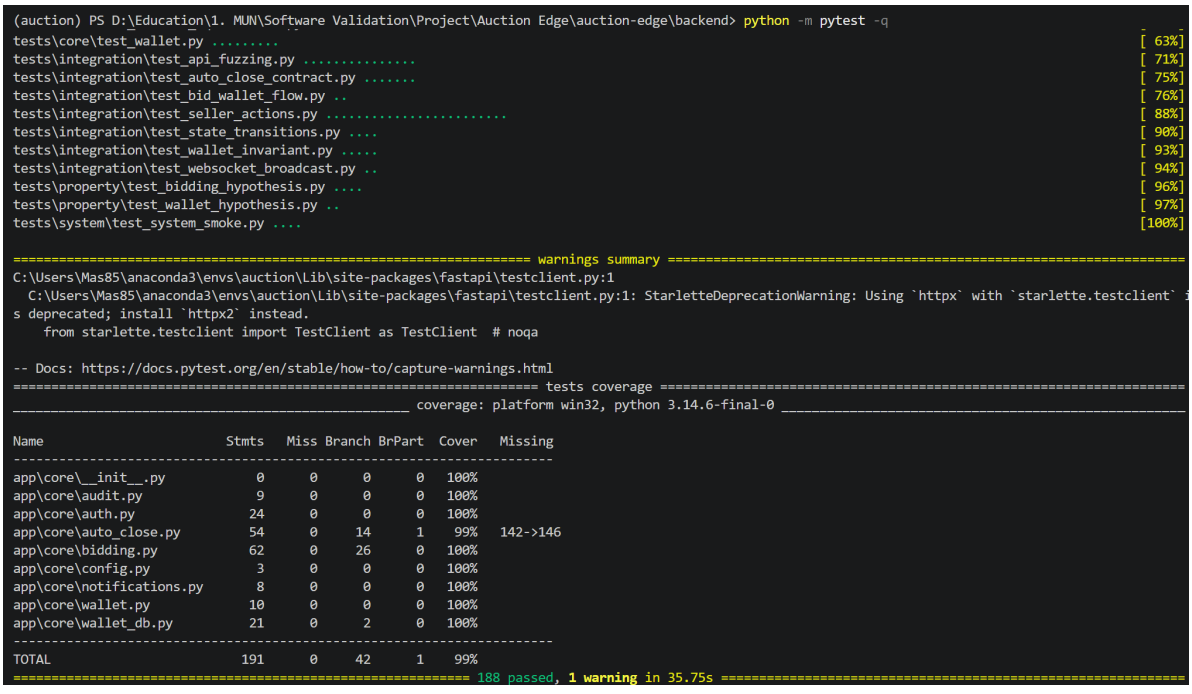

Figure 2: Full suite execution with the system tier included: 188 passed, 99% overall branch coverage on `core/`.

Coverage is not uniform across every file, and that is by design: `core/auto_close.py` sits at 99%, with exactly one branch — a defensive guard whose condition is provably unreachable given the calling function's own contract, confirmed by tracing through the code rather than assumed — left untested on purpose, rather than forced to 100% with an artificial call sequence that would misrepresent whether that guard is actually reachable.

6.2 RQ1 and RQ2: Race Conditions and Mutation Adequacy

Table 2 lists the concurrency-correctness claims checked for RQ1; Figure 3 shows the tier running.

Table 2: RQ1 claims and their verifying tests.

Claim	Verification
No lost updates under concurrent bidding	5 real OS threads bid simultaneously; final price always equals the highest accepted bid
Double-spend guard holds under real concurrency	Same bidder, two auctions, combined bid value exceeds balance; exactly one bid succeeds
True-timestamp ties resolve deterministically	Injected identical-timestamp bids resolve to exactly one winner
Auto-close is idempotent under a repeated sweep	Closing the same expired auction twice produces exactly one closure log entry

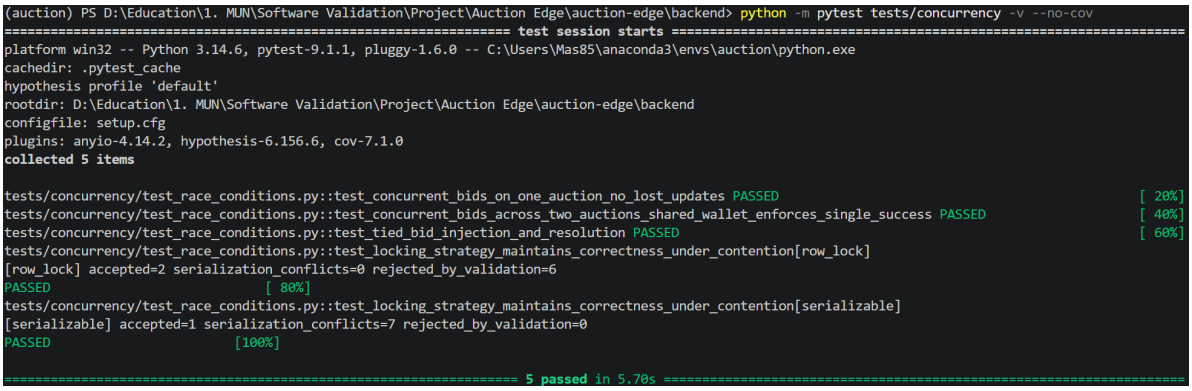

Figure 3: Concurrency tier execution (5 tests). The printed lines under the fourth and fifth test entries are the RQ4 comparative data, discussed in Section 6.4.

RQ2's mutation testing result is about as clean as this technique gets: 100 mutants were generated against `core/bidding.py` (91) and `core/wallet.py` (9), and all 100 were killed by the existing test suite, with zero survivors (Figure 4). That matters because coverage alone does not prove much on its own — a fully branch-covered line can still hide an undetected fault if nothing actually checks the value it produces. Mutation testing is the technique built specifically to catch that gap, and here it found none.

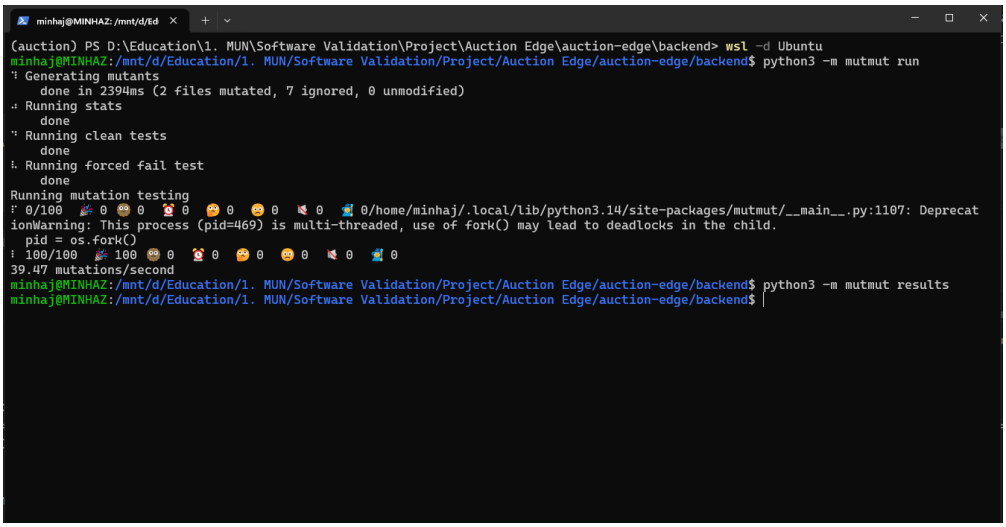

Figure 4: Mutation testing execution and result: 100/100 mutants killed, 0 survivors.

6.3 RQ3: Automated Edge-Case Generation

All six property-based tests passed with zero falsifying examples found across roughly 100 generated cases each (Figure 5), covering the bid-validation, Buy-It-Now, retraction-monotonicity, soft-close, and wallet-invariant properties described in Section 5.3. Finding no counterexample is evidence the properties hold across the distribution Hypothesis actually generated, not proof they hold for every possible input — a distinction picked up again in Section 8.

6.4 RQ4: Locking Strategy Comparison

Table 3 and Figure 6 present the RQ4 result: an identical eight-bidder contention scenario, run under both locking strategies. Both strategies kept the system correct — in every

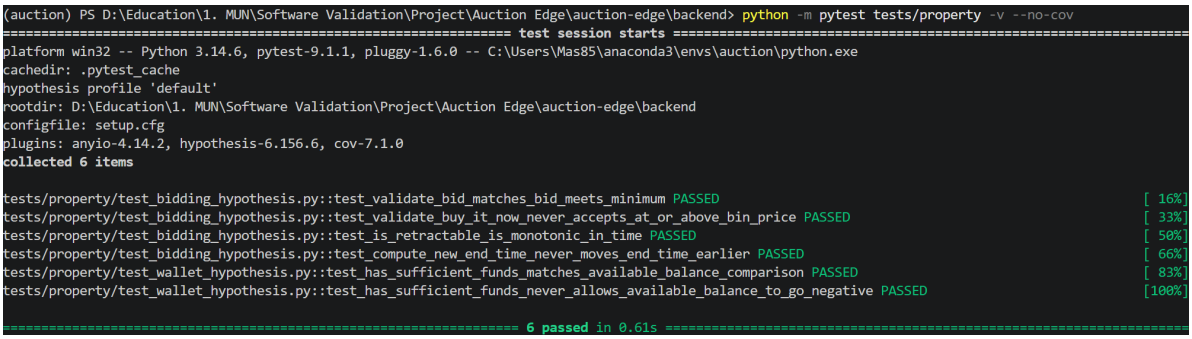

Figure 5: Property-based testing tier execution (6 tests, RQ3).

observed run, exactly two bids were accepted in total, and the final state (current price, single active bid) matched the highest accepted amount either way. Where the two strategies differ is *mechanism*. Application-level locking (`row_lock`) rejected every losing transaction cleanly through ordinary validation, producing zero serialization conflicts across repeated runs. Database-level `SERIALIZABLE` isolation instead let losing transactions pass validation and aborted them at commit time, producing between 4 and 6 serialization conflicts (out of 6 losing attempts) across repeated runs of the identical scenario. That variability is itself a real result, not noise: it reflects genuine thread-scheduling non-determinism — which transaction happens to reach the commit point first changes from run to run — whereas the `row_lock` strategy's queuing behaviour produced the same zero-conflict outcome every time it was observed.

Table 3: RQ4 comparative result: identical 8-bidder contention scenario.

Strategy	Accepted	Serialization Conflicts	Clean Rejections
`row_lock` (default)	2	0 (every run)	6
`serializable`	2	4–6 (across runs)	0–2

The practical takeaway is a genuine engineering trade-off, not one strategy simply beating the other. `row_lock` trades throughput — contending transactions queue and wait their turn — for implementation simplicity, since no caller-side retry logic is needed. `serializable` trades that queuing delay for higher potential throughput, but only if the caller actually detects and retries an aborted transaction — something this system's HTTP layer does not

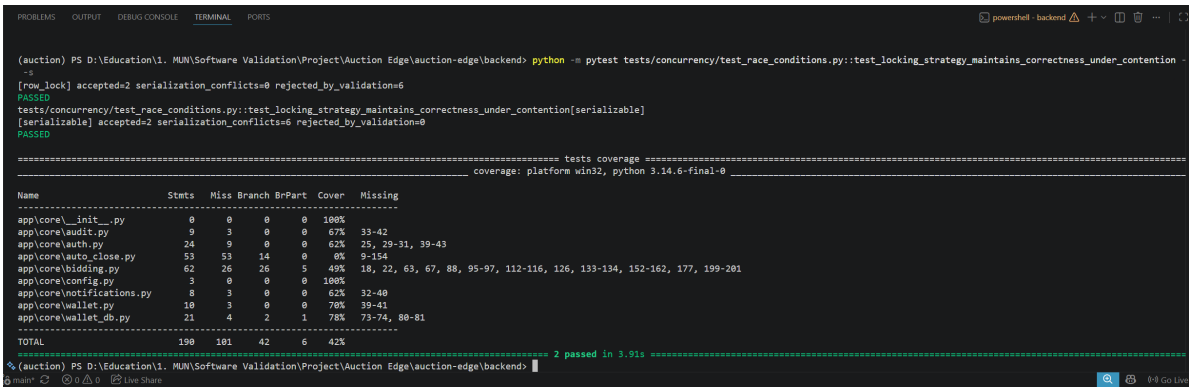

Figure 6: RQ4: isolated execution of the locking-strategy comparison test, both parametrizations.

currently do. In practice, that means a `serializable`-mode conflict is, today, simply a bid the end user silently loses, rather than one that gets transparently retried for them.

6.5 System Testing

The system-testing tier (Figure 7) validated the complete deployed configuration — three Docker containers communicating over a real network — through genuine HTTP requests, including a full register-item-auction-bid-verify flow and a regression check confirming the NaN/Infinity fix described in Section 7 holds in the actual deployed container, not merely under the in-process test client used elsewhere in the suite.

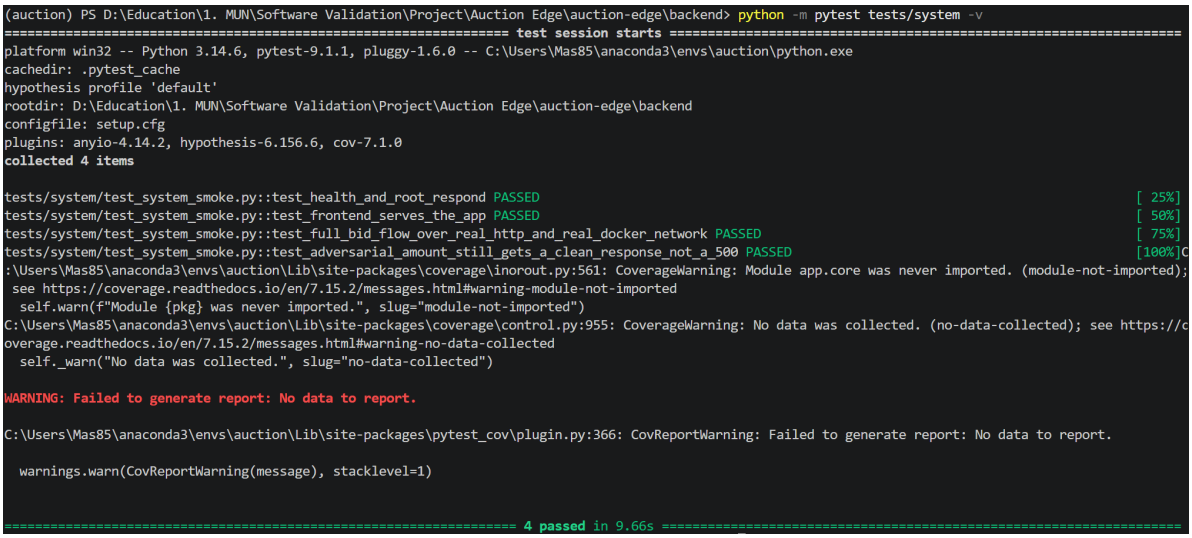

Figure 7: System testing tier execution against the full Docker Compose stack (4 tests).

6.6 Exploratory Testing Results

The four tours described in Section 5.7 produced 23 individual checks against the live frontend, summarized in Table 4. Twenty-one came back clean. Two did not.

Table 4: Exploratory tours run against the live frontend.

Tour	Checks	Result
Standard User	7	6 clean; 1 defect (Finding 3)
Saboteur	5	All clean
Antisocial	7	6 clean; 1 defect (Finding 4)
Obsessive-Compulsive	4	All clean
Total	**23**	**21 clean, 2 defects found**

The first defect: the Dashboard's "Pay Now" button never disappears once a bid has actually been paid for. It only checks whether the bid itself is still marked "active"; it has no idea whether the parent auction has closed or already been charged, since that information is not even part of the response the dashboard fetches. Clicking the button again would not double-charge anyone — the backend's own `/pay` endpoint correctly rejects a second payment attempt — but a buyer who just paid gets no visible sign that anything actually happened.

The second defect: an auction title of 300 characters with no spaces breaks the item-detail page. The Browse page's card handles the same title fine, since it truncates long text with an ellipsis; the item-detail page's heading has no equivalent protection, so the unbroken string forces the whole page to render nearly four times wider than the viewport.

Both defects are frontend-only, and neither would ever show up in any of the backend-focused tiers described above — every HTTP response involved in both cases is completely correct. What is wrong is what the React app does with a correct response once it has one, and that is a gap only frontend-facing testing can see.

One measurement along the way is worth reporting for what it says about testing methodology in general, not about the app. An early attempt to check whether the bid button shows a loading state under a slow network used a blocking `time.sleep()` call inside a Playwright

network handler, and that call stalled the automation driver's own thread — so the check actually ran only after the artificial delay had already finished, and it looked like a bug that was not really there. Re-run with a delay-free check taken immediately after the click (valid, since the button's loading state is set synchronously in the click handler, before the network call is even sent), the loading state showed up correctly. This near-miss is reported here on purpose: a slow or badly-instrumented test can manufacture a false result just as easily as a real bug can hide from a careless one.

7 Discussion and Findings

Four genuine defects came out of this project, and together they make a simple point: different testing techniques catch genuinely different kinds of fault. None of the four would have been caught by running the other three techniques harder.

Finding 1: a crash on input that is technically valid JSON. HTTP-boundary fuzzing — deliberately sending adversarial values, including the non-standard but JSON-parseable literals `NaN`, `Infinity`, and `-Infinity`, directly at the bid-placement endpoint — showed that any numeric field in the API would crash the server with an unhandled 500 error instead of a clean validation response. The cause was two independently reasonable decisions colliding: FastAPI's default validation-error handler echoes the rejected value back in its response, and the underlying ASGI framework's JSON encoder is configured for strict RFC 8259 compliance (`allow_nan=False`), which raises an uncaught exception when asked to serialize a NaN or infinite float. Pydantic's own schema validation correctly flagged the input as invalid; the crash happened afterward, while trying to report that rejection. Neither the schema validation nor the property-based testing tier (which only exercises pure Python functions, never the HTTP serialization layer) could have surfaced this fault — it took a test operating specifically at the HTTP request/response boundary. The defect was fixed with a custom exception handler that sanitizes non-JSON-safe floating-point values before

response encoding, and the fix was checked both in the in-process test suite and, separately, against the real deployed system-testing tier.

Finding 2: a code path nothing else in the suite ever touches. Every other test in this project creates user accounts by inserting a database row directly, bypassing the real `/auth/register` HTTP endpoint for speed and simplicity. The system-testing tier was the first test to exercise real user registration over real HTTP, and it revealed something none of the other tiers could have: the placeholder email domain used pervasively elsewhere in the suite (`@example.test`) is rejected by the email-validation library as an IANA special-use reserved top-level domain. That is correct behaviour on the library's part, but no other tier in the suite calls the real registration code path at all, so no other tier could ever have found it. This is a concrete argument for why a distinct system-testing tier is not redundant with a thorough integration-testing tier: the two exercise genuinely different code, even against what looks like "the same" feature on paper.

Finding 3: a payment button that does not know payment happened. Exploratory testing against the live frontend (Section 6.6) found that the Dashboard's "Pay Now" button never disappears once a bid has actually been paid for, because it only checks whether the bid is still marked "active" and has no visibility into whether the auction itself has closed or been charged. Clicking it again is harmless — the backend already rejects a duplicate payment — but the button gives a buyer no visible confirmation that their purchase went through.

Finding 4: one very long word breaks the item page. Also from exploratory testing: a 300-character auction title with no spaces breaks the item-detail page's layout, forcing the entire page to render nearly four times wider than the screen. The Browse page's card handles the identical title without any trouble, since it truncates long text with an ellipsis; the item-detail heading has no equivalent protection. A real user is unlikely to type a title quite this long by hand, but the fix costs one CSS class, and it is exactly the kind of input a hand-written test case is easy to forget to try.

Findings 3 and 4 make the same point as 1 and 2, from the other side of the stack: no amount of backend HTTP testing was ever going to catch either one, because from the API's point of view, every response involved was completely correct. What was wrong is what the React app did with a correct response after receiving it — a blind spot only frontend-facing testing can see.

A fifth, non-defect observation is worth noting separately, since it is architectural rather than a bug: as documented in Section 5.5, the system's current single-worker design, combined with synchronous database calls inside an `async` endpoint, means that concurrent HTTP load against a single running instance would not, in fact, produce genuine database-level lock contention. Correctness under real concurrent load depends on running multiple worker processes — a deployment consideration outside the scope of the endpoint code itself, but material to any claim about this system's production concurrency behaviour.

8 Threats to Validity and Limitations

A few things limit how far this project's claims can be stretched. Mutation testing was deliberately scoped to two files; that scoping is defensible (Section 5.2), but it means the 100% mutation-kill result does not extend to the rest of the codebase — the database-orchestration and API layers were validated by other techniques (contract-based and integration testing), not by mutation analysis. Property-based testing finding no counterexample is evidence the tested properties hold across the distribution Hypothesis actually generated, not proof they hold universally; Hypothesis's generation strategies were deliberately bounded (e.g. excluding unbounded positive numbers from the adversarial generator specifically because they are legitimately valid bids, not because they are out of scope), and a differently-distributed generator could in principle surface different behaviour. The concurrency and system-testing tiers were both executed on a single development machine; a genuine multi-host network partition or clock-skew scenario was never exercised. The exploratory tours in Section 6.6

were driven through Playwright automation — and one of those tours produced an instructive near-miss, described in Section 6.6, where a blocking call inside the automation itself briefly manufactured a false reading. Finally, formal specification — originally considered as a candidate technique for the auto-close atomicity and wallet-hold invariants — was not completed within this project's timeline, and is noted here as future work rather than claimed as delivered.

9 Conclusion

This project shows that a layered, multi-technique V&V strategy works better for a concurrency-critical system than any single technique applied thoroughly, for a simple reason: different techniques structurally cannot see the same faults. Mutation testing confirmed the existing suite actually catches its highest-value boundary comparisons (RQ2: 100/100 killed). Property-based testing confirmed four core invariants across a generated input space well beyond what a hand-authored boundary table would practically cover (RQ3). A purpose-built concurrency harness, engineered specifically around an architectural constraint that would otherwise have invalidated the experiment, produced a genuine, reproducible comparison between two concurrency-control strategies (RQ4). HTTP-boundary fuzzing and system testing each surfaced a real backend defect that no other tier in the suite could structurally have found, and a round of exploratory testing against the live frontend caught two further, purely UI-side defects that no backend-focused tier could ever have seen. The result is a 188-test suite at 99% branch coverage and 100% mutation adequacy, and four real bugs found and fixed along the way. Measured this carefully, test adequacy is a stronger and more specific claim than test coverage measured alone.

References

[1] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, Apr. 1978.

[2] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, Sept.–Oct. 2011.

[3] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. 5th ACM SIGPLAN Int. Conf. Functional Programming (ICFP '00)*, 2000, pp. 268–279.

[4] J. C. King, "Symbolic execution and program testing," *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, July 1976.

[5] D. MacIver, Z. Hatfield-Dodds, and Contributors to Hypothesis, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA: Morgan Kaufmann, 1993.

[7] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, "A critique of ANSI SQL isolation levels," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1995, pp. 1–10.

[8] J. A. Whittaker, *Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design*. Upper Saddle River, NJ: Addison-Wesley, 2010.

[9] G. J. Myers, C. Sandler, and T. Badgett, *The Art of Software Testing*, 3rd ed. Hoboken, NJ: Wiley, 2011.

[10] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2016.